

ニューラルネットワークの最小単位

`fit()` の中で境界が動く

rkskmt

1

ここまでの道具

- **勾配降下法**：傾きの逆方向に少しずつ進んで最小値を探す（最適化の回）
- **回帰の `fit()` は自作済み**：体長→重さの直線を勾配降下で当て、sklearn と一致した（前回）
- 残るは**分類**：ロジスティック回帰の **`fit()`** は、決定境界の回からブラックボックスのまま

① 今回の問い

同じ道具で、**分類の `fit()`** も開けられるか？ — 開けると、そこに**ニューラルネットワークの最小単位**が現れる

2

最小単位の構造

3

入力から出力まで、たった2ステップ

Step 1：アフィン変換（affine transformation）

$$z = \alpha_1 x_1 + \alpha_2 x_2 + b$$

- x_1, x_2 ：入力（輝度、重さ）
- α_1, α_2 ：**重み**（どの特徴量をどれだけ重視するか）
- b ：**バイアス**（全体のオフセット）
- z ：スカラー1個

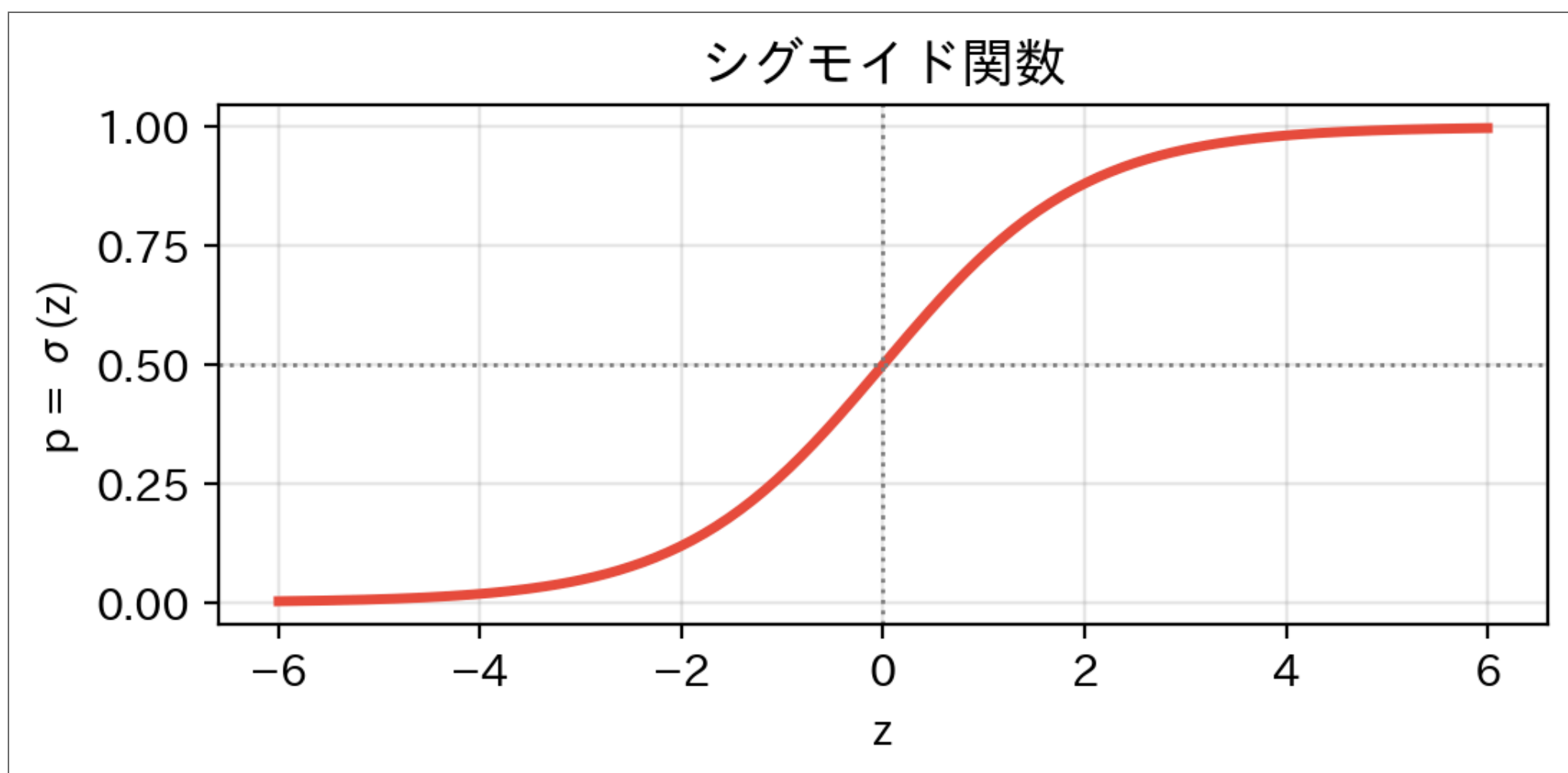
幾何的な意味

- 2次元の点を α 方向に**射影**して1次元に落とす
- b で位置を調整する

4

Step 2：シグモイドで「確率」にする

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$



- 実数全体 $(-\infty, +\infty)$ を **0~1 の範囲** に押し込む
- $p \geq 0.5 \rightarrow$ Salmon、 $p < 0.5 \rightarrow$ Sea bass と判定
- $z = 0$ (=決定境界の上) のとき、ちょうど $p = 0.5$

5

実はロジスティック回帰

- 「アフィン変換 → シグモイド」 — この2ステップが**ニューラルネットワークの最小単位**（1個のニューロン）
- そして、これは**ロジスティック回帰そのもの**
- **sklearn** の **LogisticRegression** が中でやっているのもこの計算

① 今回の作戦

- NN は、この最小単位の**組み合わせ**でできている
- だから、まず**1個だけ**を完全に理解する

6

学習 = 境界を動かすこと

7

何を最小化するか：損失関数

- 「いまのパラメータがどれだけダメか」を1つの数字にする

$$L = -\frac{1}{N} \sum_i [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

- 交差エントロピー (cross entropy)** と呼ぶ
- 例：正解 $y = 1$ (Salmon) なのに $p = 0.4$ しか出せない
→ $L = -\log(0.4) \approx 0.92$ (大きい=ダメ)
- 自信を持って正解 ($p = 0.99$) なら $-\log(0.99) \approx 0.01$ (小さい=良い)

📢 クイズ

知りたいのは正答率。なぜ**正答率そのもの**を最大化しない？

- 正答率はパラメータを少し動かしても**階段状にしか変化しない** → 微分が 0 か未定義
- 勾配降下法が使えない。**滑らかな代理**として交差エントロピーを最小化する

8

勾配が驚くほど簡単になる

- 損失 L を z で微分すると、連鎖律 (**chain rule**) で

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial p} \cdot \frac{\partial p}{\partial z} = \left(-\frac{1}{p}\right) \cdot p(1 - p) = p - 1 \quad (\text{正解 } y = 1 \text{ のとき})$$

- 正解が 0 のときは $p - 0$ 。つまり **常に**

$$\frac{\partial L}{\partial z} = p - y$$

- 各パラメータの更新式 (学習率 η) :

$$\alpha \leftarrow \alpha - \eta(p - y)x, \quad b \leftarrow b - \eta(p - y)$$

📖 ノート

シグモイドの微分 $p(1 - p)$ と交差エントロピーの微分 $-1/p$ が**打ち消し合う**ようにできている。「予測と正解の差 ($p - y$) を、入力の大きさに応じて配る」だけ。

9

実装：必要な関数は4つだけ

▶ 魚データの定義（クリックして展開）

```
1 def sigmoid(z):
2     return 1 / (1 + np.exp(-z))
3
4 def forward(X, alpha, b):
5     """アフィン変換 → シグモイド"""
6     return sigmoid(X @ alpha + b)
7
8 def loss_fn(y_true, p):
9     """交差エントロピー"""
10    eps = 1e-10
11    p = np.clip(p, eps, 1 - eps)
12    return -np.mean(y_true * np.log(p) + (1 - y_true) * np.log(1 - p))
13
14 def accuracy(y_true, p):
15    return np.mean((p >= 0.5) == y_true)
```

- **forward** が予測、**loss_fn** が採点。勾配は $(p - y)x$ なので関数すら要らない

10

魚を1匹ずつ見せて学習する

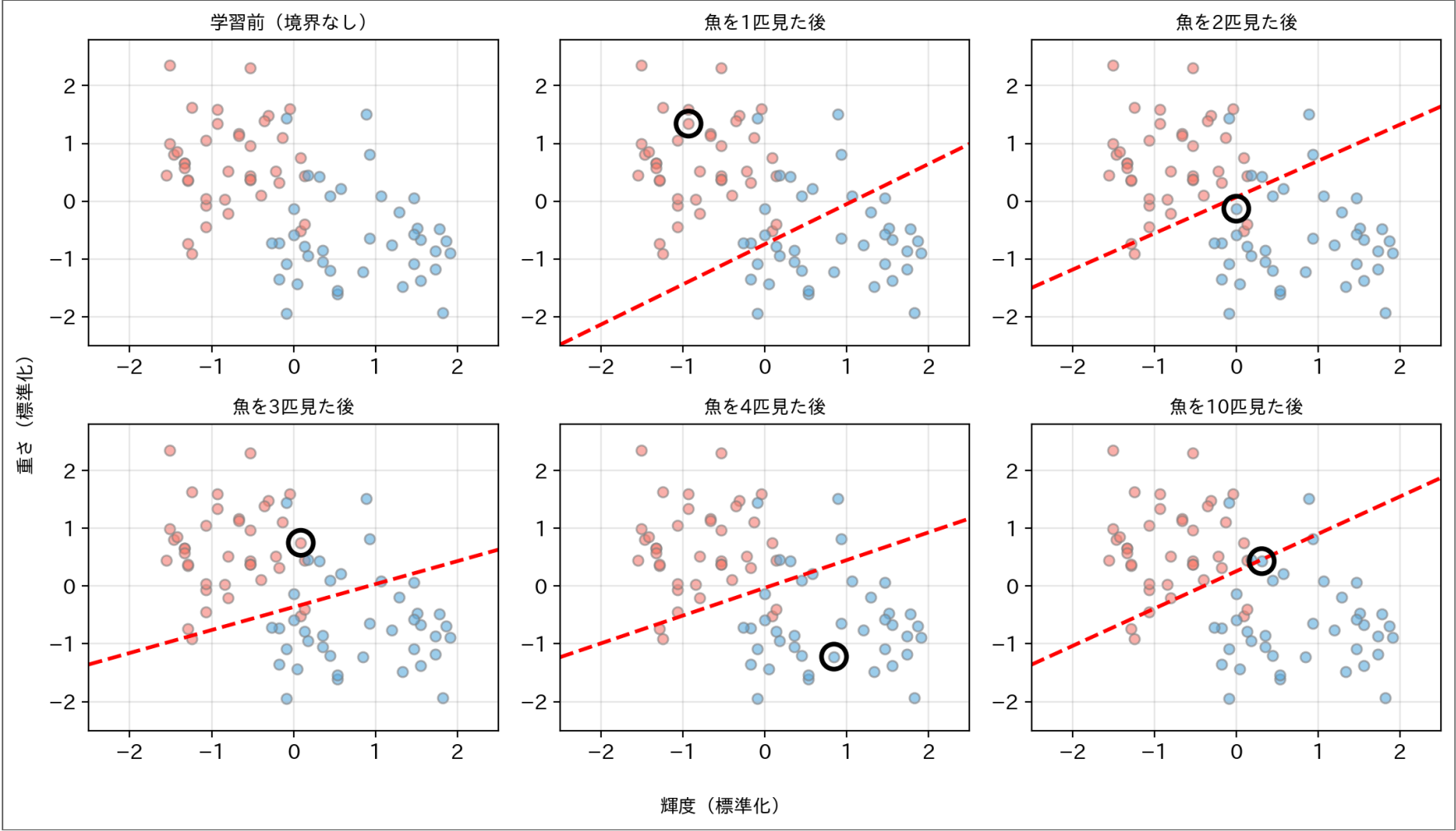
```
1 alpha = np.array([0.0, 0.0]) # 重みはゼロからスタート
2 b = 0.0
3 lr = 0.5 # 学習率
4
5 for i in range(10):
6     xi, yi = X_norm[i], y[i]
7     p_i = sigmoid(xi @ alpha + b) # Forward: 1匹だけ予測
8     delta = p_i - yi # 予測と正解の差
9     alpha = alpha - lr * delta * xi # 勾配降下で更新
10    b = b - lr * delta
11
12    p_all = forward(X_norm, alpha, b)
13    print(f"{i+1}匹目 | Loss: {loss_fn(y, p_all):.4f} | 正答率: {accuracy(y, p_all):.1%} "
14          f"| α={alpha[0]:6.3f}, {alpha[1]:6.3f} | b={b:6.3f}")
```

1匹目		Loss: 0.5326		正答率: 85.0%		α=[-0.233, 0.335]		b= 0.250
2匹目		Loss: 0.5179		正答率: 90.0%		α=[-0.233, 0.371]		b=-0.026
3匹目		Loss: 0.4974		正答率: 85.0%		α=[-0.213, 0.537]		b= 0.196
4匹目		Loss: 0.4269		正答率: 88.8%		α=[-0.359, 0.748]		b= 0.023
5匹目		Loss: 0.4044		正答率: 85.0%		α=[-0.394, 0.914]		b= 0.135
6匹目		Loss: 0.3994		正答率: 87.5%		α=[-0.360, 1.054]		b=-0.059
7匹目		Loss: 0.3901		正答率: 87.5%		α=[-0.379, 1.137]		b=-0.023
8匹目		Loss: 0.3617		正答率: 87.5%		α=[-0.516, 1.206]		b=-0.102
9匹目		Loss: 0.3364		正答率: 88.8%		α=[-0.650, 1.272]		b=-0.002
10匹目		Loss: 0.3330		正答率: 90.0%		α=[-0.741, 1.146]		b=-0.294

- 1匹見るたびにパラメータが少し動く
- たった10匹で正答率 90% まで来た

11

境界が動く様子を見る



- ○印＝直前に見せた魚。間違えるたびに境界がその魚の方へ動く

12

学習されたパラメータを読む

- 最終的な重み： $\alpha \approx [-0.74, 1.15]$

パラメータ	値	意味
α_1 (輝度)	負	暗いほど Salmon 寄り
α_2 (重さ)	正	重いほど Salmon 寄り

- 1次元分類・2次元分類の回に **目で見ても** 発見したことを、機械が数値として再発見した

i ノート

`sklearn` の `fit()` がやっているのは、本質的にこのループ（の高速・安定版）。**ブラックボックスが1つ開いた。**

13

まとめ

- NN の最小単位 = **アフィン変換** → **シグモイド** = ロジスティック回帰
- 学習 = 交差エントロピーを **勾配降下法** で最小化
 - 勾配は $(p - y)x$ — シグモイド + 交差エントロピーの組み合わせの妙
- `fit()` の中身は「予測 → 差を測る → 境界を少し動かす」の繰り返しだった

i 次回予告

この最小単位、どれだけ学習しても **直線** しか引けない。直線では絶対に分けられないデータが来たらどうする？

14